

RELAZIONE BOMBERMAN

Progetto di Programmazione
A.A. 2025/2026

Simone Coppola
Vladimir Covaci

Descrizione

Il progetto riguarda la realizzazione di un gioco ispirato a Bomberman, sviluppato in C++ utilizzando la libreria ncurses.

Il gioco comprende 5 livelli, dove il giocatore deve muoversi all'interno della mappa e piazzare bombe per distruggere muri e uccidere nemici prima dello scadere del tempo. Inoltre, è possibile raccogliere item per ottenere diversi potenziamenti.

Il giocatore utilizza le frecce direzionali o WASD per muoversi e la barra spaziatrice per piazzare le bombe.

Il programma include anche un menu e la classifica dei punteggi ottenuti dai vari giocatori.

Video Demo:

<https://drive.google.com/file/d/1fuBDblPb8s7ytOCegDEB5K0X785ECp5P/view?usp=sharing>

Divisione del lavoro

Simone Coppola

- Tempo
- Gestione delle celle della mappa
- Spawn dei nemici
- Giocatore
- Nemici
- Bombe
- Collisioni

Vladimir Covaci

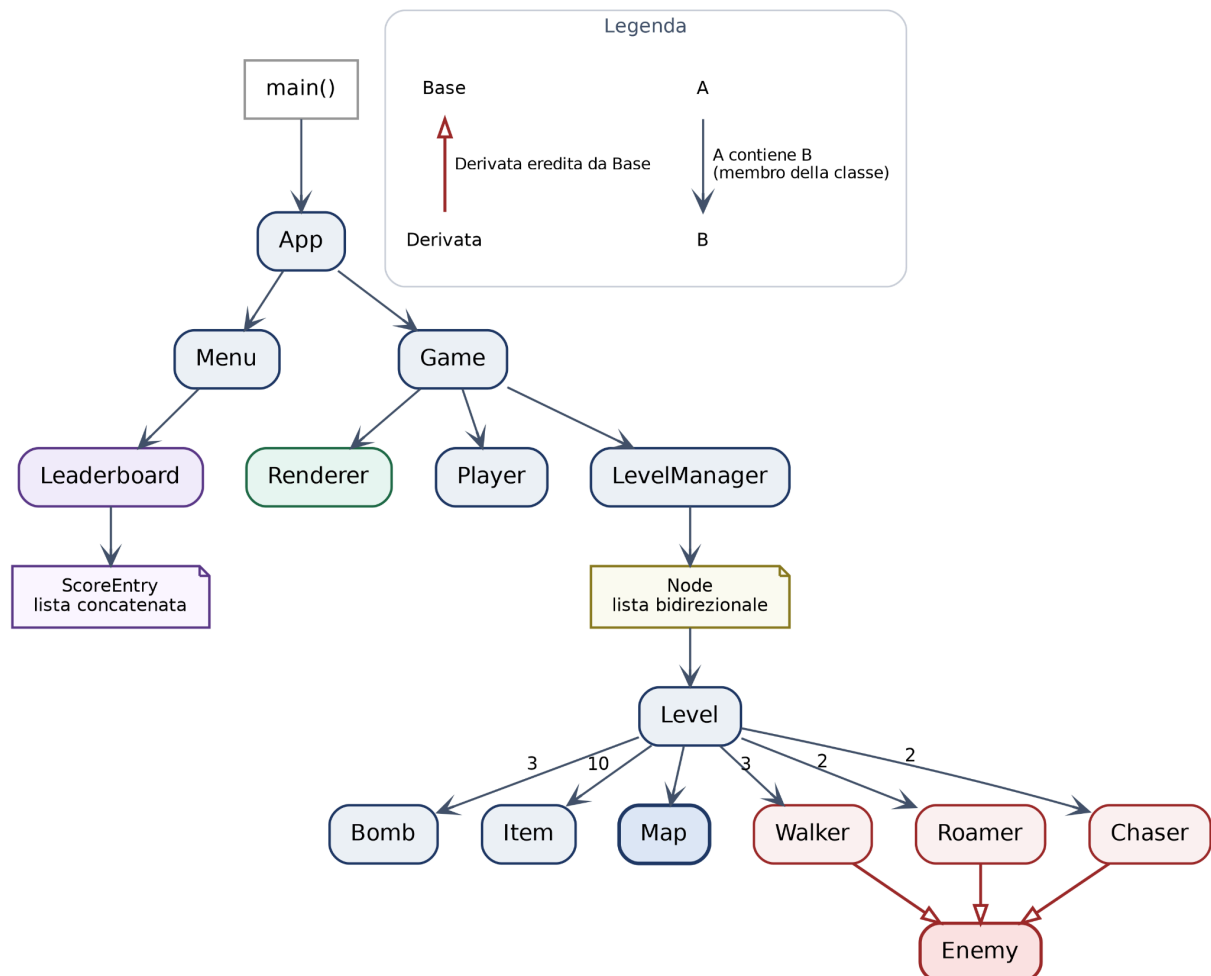
- Generazione dei muri della mappa
- Item
- Lista dei livelli
- Classifica
- Menu
- Punteggio
- Implementazione per Windows

Comune

- Grafica
- Relazioni tra classi

Indice

1. Game loop
2. Tempo
3. Mappa
4. Giocatore
5. Nemici
6. Bombe
7. Item
8. Livelli
9. Collisioni
10. Punteggio
11. Classifica
12. Menu
13. Grafica
14. Implementazione per Windows
15. Utilizzo degli LLM (Simone Coppola)
16. Utilizzo degli LLM (Vladimir Covaci)



1. Game loop

Il programma è organizzato su due livelli di loop: menu e gioco.

Il main loop della classe App gestisce il flusso generale dell'applicazione: mostra il menu, avvia una nuova partita quando richiesto e al termine permette di tornare al menu.

Durante una partita, `Game::run()` esegue il vero e proprio game loop. A ogni iterazione viene prima letto l'input, poi aggiornato lo stato del gioco e infine effettuato il rendering. In `update()` vengono aggiornati il timer, il buff del giocatore, le porte e gli elementi del livello; successivamente vengono gestite le collisioni e verificati gli stati di vittoria o sconfitta. Il rendering avviene dopo, così da mostrare gli aggiornamenti.

Il ciclo viene eseguito a intervalli regolari tramite la funzione `napms(Delay)`, ottenendo un numero approssimativo di frame al secondo determinato da `Delay`. Al termine della partita, `Game::run()` assegna eventualmente il tempo rimanente come bonus e libera esplicitamente le risorse utilizzate.

2. Tempo

Il gioco utilizza un `Delay` di 50 millisecondi, utilizzato dalla funzione `napms()` di `ncurses` per introdurre una pausa tra un frame e il successivo (durata di un tick).

Inoltre viene definita una seconda costante:

```
TICKS_PER_SECOND = 1000 / Delay = 1000 / 50 = 20
```

Tale costante viene utilizzata come riferimento per i timer degli oggetti di gioco, permettendo di esprimere le loro durate in secondi anziché in tick. Per esempio, per impostare una durata di 2 secondi, il timer viene inizializzato a $2 * \text{TICKS_PER_SECOND}$, ovvero 40 tick.

L'aggiunta di quest'ultima costante è stata essenziale per la gestione del tempo di gioco. Infatti, inizialmente ho provato a usare il tempo reale con la libreria `chrono`, perché lo consideravo più preciso e anche più chiaro semanticamente: un timer impostato a 2.0 secondi è molto più significativo di uno di 40 tick.

Tuttavia, l'utilizzo del tempo reale si è rivelato poco adatto alla struttura del gioco. Per ogni evento temporizzato era necessario memorizzare il momento in cui questo ha avuto inizio e calcolare successivamente il tempo trascorso. Per esempio, in caso di una bomba con un timer di 2 secondi, dovevo salvare l'istante in cui l'avevo piazzata e a ogni aggiornamento confrontarlo con il tempo corrente per verificare se fossero trascorsi 2 secondi. Quindi ogni oggetto con un comportamento temporizzato doveva mantenere informazioni relative al tempo di inizio di un determinato evento, ricevere il tempo corrente e gestire autonomamente il calcolo della sua durata.

La situazione si è complicata ulteriormente quando mi sono reso conto che le specifiche richiedevano il "congelamento" di un livello (quindi anche di tutti i timer associati a esso) all'uscita da esso. Di conseguenza, una bomba piazzata dovrebbe fermare il proprio timer e riprendere il conto alla rovescia solo al ritorno nel livello. Quindi, con un sistema basato sul tempo reale sarebbe necessario tenere conto anche dei periodi di pausa nel calcolo del tempo trascorso. Invece, utilizzando i tick il problema può essere risolto in maniera molto semplice: il timer della bomba viene decrementato solo durante gli aggiornamenti effettivi del livello, quindi quando quest'ultimo viene congelato anche il timer si ferma automaticamente.

Per questo motivo ho scelto di utilizzare i tick come unità di misura del tempo di gioco. Sebbene siano meno precisi del tempo reale, la loro implementazione è più naturale e si integrano meglio con il funzionamento del gioco, in particolare con il meccanismo di congelamento dei livelli. Per rendere il loro utilizzo più intuitivo, ho introdotto la costante `TICKS_PER_SECOND`.

Il giocatore ha 1000 secondi di tempo per completare tutti i livelli e vincere la partita.

3. Mappa

La mappa è stata implementata tramite la classe `Map` e rappresenta lo stato della griglia di gioco. Oltre a memorizzare il tipo di ogni cella, la classe gestisce la generazione dei muri, le posizioni disponibili per lo spawn dei nemici e lo stato delle esplosioni.

La mappa ha dimensioni 21×41 (righe x colonne).

Tipi di cella

Ogni posizione della mappa può contenere uno dei valori definiti dall'enum `Cell`:

- `NONE`: cella non valida, indica una posizione fuori dalla mappa
- `EMPTY`: cella vuota
- `WALL_SOLID`: muro solido, non può essere distrutto
- `WALL_DESTRUCTIBLE`: muro distruttibile, può essere distrutto dalle esplosioni
- `DOOR_PREV`: porta per accedere al livello precedente (in alto a sinistra)
- `DOOR_NEXT`: porta per accedere al livello successivo (in alto a destra)
- `BOMB`: bomba

Gestione delle celle

La classe `Map` fornisce metodi per verificare lo stato delle celle e modificarlo durante il gioco. In particolare, permette di controllare la presenza di muri, porte, bombe ed esplosioni e di modificare le celle in seguito agli eventi di gioco.

Muri solidi

Prima il bordo: quattro cicli che riempiono di `WALL_SOLID` la prima e l'ultima riga, la prima e l'ultima colonna. Serve perché i livelli devono essere stanze chiuse.

Poi i muri interni. Qui la regola è che una cella può diventare solida solo se $y \% 2 == 0$ & $x \% 2 == 0$, e in quel caso lo diventa a testa o croce.

Questa condizione sulla parità è la cosa più importante di tutta la generazione, anche se sembra un dettaglio. Garantisce che qualsiasi cella con almeno un indice dispari resti libera per sempre, quindi la mappa non si spezza mai in zone isolate. All'inizio avevo provato a mettere i muri solidi a caso ovunque: veniva fuori qualche livello con un angolo murato che non si poteva raggiungere in nessun modo, e visto che il livello si completa solo uccidendo tutti i nemici, se un nemico finiva lì dentro il gioco si bloccava.

Muri distruttibili

Un ciclo su tutte le celle interne: se la cella è vuota, con probabilità `percentage` diventa un muro distruttibile. La percentuale la decide il costruttore:

```
place_destructible_walls(BASE_WALL_PERCENTAGE + difficulty * 5);
```

Con `BASE_WALL_PERCENTAGE` a 5 si va dal 10% del primo livello al 30% del quinto. Il numero l'ho cambiato tre volte: il valore iniziale era troppo basso e il primo livello sembrava una stanza vuota, poi l'ho alzato troppo e al quinto livello quasi metà mappa era muro. 5 è quello che alla fine gioca meglio.

Alla fine della generazione forzo a vuote otto celle, quattro per ogni angolo in alto.

```
// Angolo sinistro (spawn iniziale e rientro dalla porta 'next')
grid[1][1] = EMPTY;
grid[1][2] = EMPTY;
grid[2][1] = EMPTY;
grid[1][3] = EMPTY;

// Angolo destro (rientro dalla porta 'prev')
grid[1][MAP_WIDTH - 2] = EMPTY;
grid[1][MAP_WIDTH - 3] = EMPTY;
grid[2][MAP_WIDTH - 2] = EMPTY;
grid[1][MAP_WIDTH - 4] = EMPTY;
```

Le prime tre di ogni gruppo sono ovvie: la cella dove nasce il giocatore e le due dove può muoversi. La quarta invece l'ho aggiunta dopo, quando ho riletto la traccia e ho visto la riga sul fatto che il giocatore deve poter piazzare una bomba senza ammazzarsi da solo. Se sto in (1,1) e piazzo una bomba, l'esplosione con raggio 1 copre (1,2) e (2,1), cioè esattamente le due caselle dove potrei scappare. Senza una quarta cella fuori dalla croce sono in trappola.

Ho scritto un programma separato che genera cinquecento mappe e conta quante volte capitava: al quinto livello succedeva in circa il 6% dei casi. Non tantissimo, ma un livello su sedici ingiocabile è comunque un problema, e la traccia lo vieta esplicitamente.

Zona sicura

La zona sicura è definita come due quadrati di lato `SAFE_ZONE_SIZE` (5 celle) posizionati negli angoli superiori sinistro e destro della mappa, uno per ogni porta. Serve per impedire lo spawn dei nemici vicino alla posizione iniziale del giocatore.

Il metodo `safe_zone()` verifica se una cella appartiene alla zona sicura.

```
bool Map::safe_zone(Position p) {
    if (p.y < 0 || p.y >= SAFE_ZONE_SIZE) {
        return false;
    }

    // Angolo sinistro
    if (p.x >= 0 && p.x < SAFE_ZONE_SIZE) {
        return true;
    }

    // Angolo destro
    if (p.x >= MAP_WIDTH - SAFE_ZONE_SIZE && p.x < MAP_WIDTH) {
        return true;
    }

    return false;
}
```

Il controllo sulla riga sta in cima perché vale per tutti e due i quadrati: per la maggior parte delle celle della mappa la funzione esce subito. Le zone sono due e non una perché nel gioco si entra in un livello da due porte diverse, e da quando il punto di rinascita dopo la morte è la porta da cui sei entrato, l'angolo destro va protetto quanto quello sinistro. Altrimenti torni indietro di un livello, muori, rinaschi addosso a un nemico e perdi un'altra vita senza aver toccato niente.

Gestione degli spawn dei nemici

Una volta costruita la mappa, il metodo `save_spawns()` costruisce l'array `spawns`, contenente le posizioni disponibili per lo spawn dei nemici. Vengono considerate solo le celle interne alla mappa che sono `EMPTY` e non appartengono alla zona sicura.

Il metodo `shuffle_spawns()` esegue lo shuffle dell'array `spawns` utilizzando l'algoritmo di Fisher-Yates. Si parte dall'ultima posizione dell'array e a ogni iterazione si sceglie casualmente un indice compreso tra 0 e quello corrente; quindi si scambiano gli elementi nelle due posizioni. La procedura termina una volta raggiunto il primo elemento dell'array.

Il metodo `get_random_spawn()` restituisce uno spawn valido (l'ultimo elemento nell'array `spawns` attualmente) e lo rimuove dalla lista dei possibili spawn (decrementando il contatore `spawn_count`). Simula quindi l'operazione di pop da una pila, usando un array. Lo shuffle effettuato all'inizio ci garantisce la casualità delle posizioni estratte.

Questa scelta permette di evitare di selezionare ogni volta un indice casuale dell'array e spostare a sinistra gli elementi successivi dopo la rimozione per mantenerli contigui. Infatti, in seguito allo shuffle, è sufficiente estrarre l'ultimo elemento.

Se le posizioni disponibili terminano, il metodo restituisce `POSITION_NONE`; tale costante, che corrisponde alla posizione `(-1, -1)`, viene utilizzata in tutto il programma per indicare una posizione non valida. Inoltre, un nemico costruito lì nasce già morto, in quanto fuori dai limiti della mappa.

Gestione delle esplosioni

A differenza delle bombe, le esplosioni non vengono memorizzate direttamente in grid, ma nella matrice `explosion`, che memorizza il numero di esplosioni presenti in ogni cella.

Viene utilizzato un contatore invece di un semplice booleano per rappresentare correttamente la sovrapposizione di più esplosioni nella stessa cella. Per esempio, se due bombe generano un'esplosione nella stessa posizione, il valore della cella diventa 2; quando una delle due esplosioni termina, il valore viene decrementato a 1 e la cella continua a essere considerata in esplosione.

I metodi `set_explosion(Position p)` e `unset_explosion(Position p)` incrementano e decrementano il contatore delle esplosioni della cella in posizione `p`, rispettivamente. `is_explosion(Position p)` considera una cella in esplosione quando il relativo contatore è maggiore di 0.

Reset

Il metodo `save_start_grid()` viene utilizzato al termine della generazione della mappa per salvare una copia della configurazione iniziale in `start_grid`.

Il metodo `reset()` utilizza questa copia per ripristinare lo stato iniziale della mappa, copiando `start_grid` in `grid` e azzerando la matrice delle esplosioni.

4. Giocatore

Il giocatore è implementato tramite la classe `Player`, che ne gestisce la posizione, le vite, il movimento e il potenziamento che modifica temporaneamente il raggio delle bombe.

Gestione dello stato

Il giocatore viene inizializzato di default in posizione (1, 1) con `MAX_LIVES` (3) vite. Inoltre, viene salvata la posizione iniziale in `start_p`, il raggio delle bombe viene inizializzato a 1 e il potenziamento viene inizialmente disattivato.

I metodi `get_position()` e `set_position(Position _p)` permettono rispettivamente di ottenere e modificare la posizione corrente, mentre `spawn(Position _p)` imposta una nuova posizione corrente e iniziale.

La gestione delle vite è affidata ai metodi `get_lives()`, `gain_life()` e `lose_life()`. Il numero non può superare `MAX_LIVES`. Il metodo `is_dead()` restituisce `true` quando il giocatore ha esaurito tutte le vite.

Ho deciso di impostare un limite massimo di vite relativamente basso a causa della presenza di item in grado di aumentarle: un limite ancora maggiore avrebbe reso la morte un evento piuttosto raro.

Movimento

Il metodo `can_move_to(Map& map, Position next)` verifica se il giocatore può raggiungere una determinata posizione. Il movimento è consentito solo se la cella di destinazione non è fuori dai limiti della mappa e non corrisponde a un muro o a una bomba.

Il metodo `move(Map& map, Direction d)` calcola la posizione successiva in base alla direzione indicata e aggiorna la posizione corrente solo se la cella di destinazione è valida.

Reset

Il metodo `reset()` ripristina lo stato del giocatore in seguito alla perdita di una vita: viene riportato nella posizione memorizzata in `start_p`, il raggio delle bombe viene reinizializzato a 1 e il potenziamento viene disattivato.

5. Nemici

I nemici sono implementati tramite una classe base `Enemy`, dalla quale derivano tre classi: `Walker`, `Roamer` e `Chaser`.

Tutti i nemici condividono la gestione della posizione, della velocità, del movimento temporizzato e dello stato di vita, mentre ogni sottoclasse implementa una diversa strategia di movimento.

In particolare:

- `Walker`: sceglie casualmente una direzione e la mantiene finché non incontra un ostacolo; a quel punto ne sceglie casualmente una nuova
- `Roamer`: sceglie casualmente una nuova direzione a ogni movimento
- `Chaser`: sceglie la direzione che minimizza la sua distanza dal giocatore

Shuffle delle direzioni

La classe base `Enemy` memorizza un array contenente le quattro direzioni (`UP`, `LEFT`, `DOWN`, `RIGHT`), utilizzato dalle classi derivate per determinare l'ordine in cui vengono considerati i possibili spostamenti. Ho preferito memorizzare le direzioni invece delle posizioni adiacenti perchè quest'ultime le avrei dovute ricalcolare ogni volta, mentre le direzioni sono fisse. Inoltre in questo modo mantengo l'implementazione del movimento dei nemici simile a quella del giocatore.

Il metodo `shuffle_directions()` mescola casualmente le quattro direzioni disponibili e viene utilizzato dai nemici che adottano una strategia di movimento casuale. La sua implementazione sfrutta l'algoritmo di Fisher-Yates, già utilizzato nella classe `Map` per gestire le possibili posizioni in cui spawnare nemici.

Velocità

La classe base `Enemy` possiede un campo `speed`, un intero che rappresenta la sua frequenza di movimento (movimenti al secondo), legato a `move_timer`, un intero che rappresenta i tick rimanenti al prossimo movimento.

Tale timer viene inizializzato nel modo seguente:

```
move_timer = TICKS_PER_SECOND / speed
```

In questo modo, aumentando la velocità diminuisce l'intervallo tra due movimenti consecutivi.

Gestione dello stato

Un nemico può essere inizializzato in una determinata posizione e con una data velocità. Inoltre, la sua posizione iniziale viene salvata in `start_p`. Una posizione pari a `POSITION_NONE` viene utilizzata per rappresentare un nemico non presente sulla mappa: in questo caso viene inizializzato direttamente come morto.

Il metodo `get_position()` permette di ottenere la posizione del nemico, `kill()` di ucciderlo e `is_dead()` di verificarne lo stato di vita.

Gestione del movimento

Il metodo `can_move_to(Map& map, Position next)` verifica se il nemico può raggiungere una determinata posizione. Il movimento è consentito solo se la cella di destinazione non è fuori dai limiti della mappa e non corrisponde a un muro, a una bomba o a un'esplosione.

Movimento (Walker)

La sottoclasse `Walker` implementa un comportamento parzialmente casuale e molto prevedibile. Memorizza, oltre ai campi ereditati dalla classe base, la sua direzione corrente, scelta casualmente all'inizializzazione.

Il metodo `move(Map& map)` tenta di muovere il nemico nella direzione corrente. Se la cella successiva è libera, il movimento viene effettuato e la direzione non cambia. Invece, se viene incontrato un ostacolo, il `Walker` utilizza `shuffle_directions()` per mescolare casualmente le quattro direzioni e sceglie la prima che gli consente di effettuare un movimento; quindi la direzione scelta diventa la nuova direzione corrente.

Il comportamento risultante è quello di un nemico che tende a mantenere la propria direzione, cambiandola solo quando non può più proseguire.

Questo è stato l'ultimo nemico che ho implementato, dopo aver provato la versione originale di Bomberman: replica infatti il comportamento dei suoi nemici base.

Movimento (Roamer)

La sottoclasse Roamer implementa un comportamento completamente casuale ed è stato il primo nemico che ho implementato.

Il metodo `move(Map& map)` utilizza `shuffle_directions()` per mescolare casualmente le quattro direzioni e sposta il nemico nella prima posizione raggiungibile.

A differenza del Walker, il Roamer non conserva la direzione del movimento precedente: a ogni movimento viene effettuata una nuova scelta casuale.

Nonostante la sua semplicità e bassa pericolosità, è il tipo di nemico che, se ucciso, permette di guadagnare più punti (10, come Chaser, contro i 5 di Walker); questo perché il suo valore risiede nella sua imprevedibilità, che lo rende più difficile da eliminare rispetto agli altri nemici.

Ordinamento delle direzioni (Chaser)

Il metodo `distance(Position a, Position b)` calcola la distanza di Manhattan tra due posizioni, considerando solo gli spostamenti orizzontali e verticali, senza tenere conto degli ostacoli presenti sulla mappa:

$$\text{distance}(a, b) = |a.x - b.x| + |a.y - b.y|$$

Il metodo `sort_directions(Position player_p)` ordina le quattro direzioni (senso crescente) in base alla distanza tra la posizione successiva del nemico e quella del giocatore.

Per l'ordinamento è stato utilizzato l'algoritmo Selection Sort. Nonostante non sia un algoritmo molto efficiente, siccome le direzioni sono solamente quattro il numero di confronti è molto basso e il costo dell'ordinamento è trascurabile.

Movimento (Chaser)

La sottoclasse Chaser implementa un comportamento completamente deterministico e orientato all'inseguimento del giocatore.

Il metodo `move(Map& map)` utilizza `sort_directions()` per ordinare le quattro direzioni in base alla distanza dal giocatore e sposta il nemico nella prima posizione raggiungibile.

Quindi il comportamento del Chaser è greedy: a ogni movimento sceglie la direzione che lo avvicina maggiormente al giocatore, senza calcolare un percorso completo verso di esso. Il limite principale di questo approccio deriva dal fatto che il Chaser valuta esclusivamente la situazione immediatamente successiva al proprio movimento, senza considerare il percorso complessivo necessario per raggiungere il giocatore.

Di conseguenza, il comportamento risulta praticamente perfetto quando tra il nemico e il giocatore sono presenti pochi ostacoli, perché in questo caso la direzione che minimizza la distanza tende a coincidere con quella che conduce verso il giocatore.

La situazione cambia quando aumenta il numero di muri presenti sulla mappa: in questi casi, anche se esiste un percorso valido per raggiungere il giocatore, il Chaser non funziona come vorrei e tende a scegliere ripetutamente mosse vantaggiose nell'immediato ma che non pagano nel lungo periodo. Questo può portarlo a rimanere bloccato in determinati schemi di movimento, per esempio arrivando a spostarsi avanti e indietro tra due posizioni senza riuscire a superare l'ostacolo che lo separa dal giocatore.

Discuto più approfonditamente questo problema nella sezione "A. Utilizzo degli LLM (Simone Coppola)", paragrafo "Miglioramento del movimento del Chaser".

Aggiornamento

Ogni classe derivata definisce un proprio metodo per aggiornarsi, ma funzionano tutti allo stesso modo: a ogni aggiornamento `move_timer` viene decrementato e, quando raggiunge 0, il nemico effettua un movimento e il timer viene ripristinato.

Reset

Il metodo `reset()` ripristina lo stato del nemico durante il reset del livello: viene riportato nella posizione memorizzata in `start_p`, `move_timer` viene reimpostato e lo stato di vita viene ripristinato in base alla posizione iniziale (un nemico inizializzato con posizione `POSITION_NONE` rimane morto anche dopo il reset).

6. Bombe

Le bombe sono implementate tramite la classe `Bomb`, che gestisce il loro intero ciclo di vita: costruzione, piazzamento, attesa, lampeggio, esplosione e reset. Il comportamento è gestito attraverso diversi timer, espressi in tick.

Il giocatore può avere al massimo tre bombe attive contemporaneamente. Per poter piazzare una nuova bomba, deve attendere che almeno una di quelle già posizionate sia esplosa e abbia liberato uno dei tre slot disponibili.

A un certo punto, per rendere il gioco più fedele all'originale, ho provato ad assegnare un solo slot per le bombe, aumentabile fino a tre raccogliendo uno specifico item. Inoltre alla perdita di una vita gli slot venivano resettati e tornavano a uno. Però, mi sono accorto che tale approccio era poco funzionale alla nostra implementazione, in quanto con un solo slot diventava difficile uccidere i nemici, soprattutto i Roamer, rendendo il gioco frustrante invece di aumentare la sfida. Inoltre, il reset era particolarmente penalizzante, soprattutto ai livelli più alti. Perciò ho deciso di tornare all'implementazione iniziale, mantenendo tre slot fissi, anche perchè ritengo che questo renda il gioco più divertente.

Campi

La classe `Bomb` è la più complessa e stratificata dell'intero programma, per cui ritengo opportuno descrivere in dettaglio ogni suo campo, per rendere più chiaro il suo funzionamento:

- `Position p`: posizione della bomba
- `int range`: raggio (massimo) dell'esplosione
- `int explosion_range[DIRECTIONS_COUNT]`: raggio effettivo dell'esplosione in ogni direzione (può essere inferiore a `range` a causa degli ostacoli presenti sulla mappa)
- `bool active`: indica se la bomba è stata piazzata e non è ancora esplosa (in attesa) oppure sta esplodendo

- `bool blink`: indica se la bomba sta lampeggiando
- `bool exploding`: indica se la bomba sta esplodendo
- `int blink_timer`: tick rimanenti all'inizio o alla fine del lampeggio
- `int exploding_timer`: tick rimanenti all'inizio dell'esplosione (dal piazzamento della bomba)
- `int explosion_timer`: tick rimanenti alla fine dell'esplosione (dall'inizio dell'esplosione)

Inoltre, la durata delle diverse fasi del ciclo di vita della bomba è definita attraverso tre costanti:

- `BLINK_TIMER_START`: durata dell'intervallo tra due cambiamenti dello stato di lampeggio (0,25 secondi)
- `EXPLODING_TIMER_START`: tempo che intercorre dal piazzamento della bomba all'inizio dell'esplosione (2 secondi)
- `EXPLOSION_TIMER_START`: tempo che intercorre dall'inizio alla fine dell'esplosione (0,5 secondi)

Viene utilizzata la costante `TICKS_PER_SECOND` come riferimento temporale.

Gestione dello stato

In seguito alla sua costruzione, una bomba attraversa principalmente tre fasi durante il suo ciclo di vita:

- **Piazzamento**: la bomba diventa attiva e viene piazzata sulla mappa
- **Attesa**: il timer dell'esplosione viene decrementato a ogni frame e la bomba alterna periodicamente il proprio stato di lampeggio
- **Esplosione**: la bomba genera l'esplosione nella propria posizione e nelle quattro direzioni; al termine di essa, tutte le celle interessate vengono ripulite e la bomba viene resettata

Lo stato della bomba può essere verificato tramite i metodi `is_active()`, `is_blinking()` e `is_exploding()`.

Il metodo `update(Map& map)` viene chiamato durante l'aggiornamento del gioco per gestire l'evoluzione temporale delle bombe attive.

Piazzamento

Il metodo `place(Map& map, Position _p, int _range)` assegna alla bomba una nuova posizione e un nuovo raggio e la imposta come attiva. Inoltre, aggiorna la mappa piazzando una bomba nella posizione indicata.

Attesa

Finché la bomba non sta esplodendo, vengono aggiornati `blink_timer` ed `exploding_timer`.

Quando `blink_timer` raggiunge 0, il valore di `blink` viene invertito e il rispettivo timer viene resettato. Quindi la bomba alterna periodicamente il proprio stato di lampeggio.

Quando `exploding_timer` raggiunge 0, viene chiamato `explode()` e la bomba passa alla fase di esplosione.

Esplosione

Il metodo `explode(Map& map)` avvia la fase di esplosione. Innanzitutto `exploding` viene impostato a `true`, viene generata l'esplosione nella posizione centrale e la bomba viene rimossa dalla mappa.

Successivamente viene chiamata `start_explosion()` per propagare l'esplosione nelle quattro direzioni, formando una struttura a croce.

Per ciascuna direzione, la propagazione prosegue fino al raggio massimo dell'esplosione, verificando a ogni passo che la cella non si trovi fuori dai limiti della mappa e non contenga un muro solido: in entrambi i casi l'esplosione si interrompe.

Se la cella è raggiungibile, viene contrassegnata come interessata dall'esplosione e il raggio effettivo viene incrementato.

Nel caso di un muro distruttibile, la cella viene coinvolta nell'esplosione, ma la propagazione si interrompe: il muro viene distrutto e assorbe l'esplosione, impedendole di proseguire nelle celle successive.

Le celle coinvolte rimangono nello stato di esplosione fino alla scadenza di `explosion_timer`; a quel punto, il metodo `stop_explosion()` utilizza i valori salvati in `explosion_range` per percorrere nuovamente le celle interessate e rimuovere l'esplosione dalla mappa.

Successivamente la bomba viene riportata allo stato iniziale tramite `reset()`.

Reset

Il metodo `reset()` termina il ciclo di vita corrente della bomba e la rende nuovamente disponibile per un nuovo piazzamento.

L'array `explosion_range` viene reinizializzato a 0 per ogni direzione, vengono disattivati gli stati `active`, `blink` ed `exploding` e tutti i timer vengono riportati ai rispettivi valori iniziali.

In questo modo lo stesso oggetto Bomb può essere utilizzato più volte durante la partita.

7. Item

Gli item si prendono camminandoci sopra e si ottengono rompendo muri o uccidendo nemici. Ce ne sono quattro tipi.

- `ITEM_RANGE`: aumenta il raggio delle bombe per un po' di tempo
- `ITEM_LIFE`: una vita in più, subito
- `ITEM_SCORE`: 5 punti in più, subito
- `ITEM_TIME`: trenta secondi in più sul timer

Quello che la traccia chiede davvero è il primo: un item che aumenta il raggio "per un tempo di 5 o 10 secondi". Le due costanti sono esattamente quelle, convertite in tick, e quando l'item viene generato la durata viene sorteggiata tra le due. Gli altri tre li ho aggiunti dopo perché con solo l'item raggio i muri rotti davano poca soddisfazione.

Come sono fatti

Ogni livello ha un array fisso di dieci Item. Non vengono mai creati o distrutti: il campo `active` distingue quelli a terra da quelli liberi. Quando ne serve uno nuovo, `spawn_item()` cerca il primo slot spento e lo riusa; se sono tutti pieni non succede niente.

È una specie di pool. L'ho fatto così soprattutto perché non possiamo usare i vector, ma anche perché non ha senso allocare e liberare di continuo oggetti che sono al massimo dieci e di dimensione nota. Dieci non lo raggiungiamo mai in pratica, ma tenere il controllo evita di scrivere fuori dall'array se un giorno alziamo le probabilità di drop.

Quando escono

Il metodo `Level::try_drop_item(Position p, int chance)` viene chiamato quando cade un muro (25%) e quando muore un nemico (50%). Se il tiro va a segno, sorteggia uniformemente uno dei quattro tipi e lo mette a terra in quella posizione. I nemici hanno il doppio delle probabilità dei muri perché sono molti meno: in un livello rompi decine di muri ma uccidi sette nemici al massimo.

La raccolta

A ogni frame `Game::handle_item_collection(Level& level)` confronta la posizione del giocatore con quella degli item attivi e applica il relativo effetto in base al tipo.

Item raggio

L'effetto dell'item raggio è gestito dal metodo `Player::apply_buff(int duration)`. Se ne raccogli un secondo mentre il primo è ancora attivo il raggio non aumenta ancora, ma la durata si somma. Ci ho pensato un po': far crescere il raggio a ogni item sembrava più divertente, ma con raggio 3 o 4 su una mappa aperta ti ammazzi da solo troppo facilmente.

Il metodo `Player::update_buff()` viene utilizzato per aggiornare il potenziamento durante il gioco. A ogni frame decrementa `buff_timer` e quando raggiunge 0 il potenziamento termina e il raggio delle bombe torna al valore base 1.

Il tempo che resta al potenziamento è scritto nel pannello a destra, sotto le vite: `EFFECT: RANGE 'N'` S e poi `EFFECT: NONE` quando finisce. All'inizio non c'era e il potenziamento era praticamente invisibile: capivi che era scaduto solo perché a un certo punto le bombe facevano meno danno. I secondi li arrotondo per eccesso, sennò l'ultimo secondo mostra zero mentre l'effetto è ancora attivo.

8. Livelli

Il gioco è composto da 5 livelli, caratterizzati da una difficoltà crescente ottenuta attraverso l'aumento della percentuale dei muri distruttibili, del numero e della velocità dei nemici. I nemici sono introdotti progressivamente nel seguente modo:

1. 3 Walker (velocità 1)
2. 3 Walker (velocità 2), 1 Roamer (velocità 1)
3. 3 Walker (velocità 2), 2 Roamer (velocità 2)
4. 3 Walker (velocità 2), 2 Roamer (velocità 2), 1 Chaser (velocità 2)
5. 3 Walker (velocità 2), 2 Roamer (velocità 2), 2 Chaser (velocità 2)

Un livello viene considerato completato quando tutti i nemici presenti in esso sono morti. A quel punto il giocatore può utilizzare le porte per accedere ai livelli successivi o tornare a quelli precedenti, se possibile.

Implementazione

La traccia chiede una lista bidirezionale per i livelli, in modo da poter andare avanti e indietro in qualsiasi momento. Ho fatto una struct Node e una classe LevelManager che la gestisce.

```
struct Node {  
    Level level;  
    Node* next;  
    Node* prev;  
};
```

Due puntatori: head sul primo nodo, current sul livello dove si trova il giocatore.

Costruzione

Il costruttore fa cinque nodi in fila. Tiene da parte il puntatore al nodo appena creato e lo usa al giro dopo per collegare entrambe le direzioni. Ogni livello riceve il proprio numero, che poi determina densità dei muri e numero di nemici.

Per liberare i nodi c'è free_levels(), che scorre la lista salvando il puntatore al successivo prima di ogni delete. La chiama Game::run() quando la partita finisce.

All'inizio questo lavoro lo faceva il distruttore ~LevelManager(), che sembrava la scelta ovvia. L'ho tolto perché con un distruttore la deallocazione avviene quando il LevelManager esce di scopo, cioè in un momento che non decido io, e la classe ha un puntatore grezzo senza costruttore di copia: se qualcuno la copiasse per sbaglio i nodi verrebbero liberati due volte. Con un metodo esplicito la liberazione avviene dove la scrivo, e si vede leggendo run().

Muoversi tra i livelli

I metodi di navigazione controllano sempre i puntatori prima di seguirli: chiamare go_to_next_level() sull'ultimo livello non fa niente invece di far crashare il gioco.

Il metodo update_doors() apre e chiude le porte guardando se esistono i nodi adiacenti. La porta di sinistra resta aperta ogni volta che c'è un livello precedente, anche se quello corrente non è completo: è una richiesta esplicita della traccia.

Cancellare un livello completato

I livelli completati devono sparire. Se ne occupa remove_current_level(bool forward). Il parametro dice da che parte sta uscendo il giocatore, e quindi su quale nodo spostare current.

Trova il nuovo nodo corrente, ricollega tra loro il precedente e il successivo saltando quello di mezzo, aggiorna la testa se serve, sposta il puntatore e libera la memoria. Ritorna un booleano che dice se la rimozione è avvenuta davvero.

C'è un controllo all'inizio che sembra ridondante ma non lo è: se il nodo adiacente nella direzione richiesta non esiste, il metodo esce senza fare niente. Serve a non svuotare mai la lista, perché altrimenti current resterebbe a puntare nel vuoto e la prima chiamata successiva farebbe saltare tutto. In pratica l'ultimo livello rimasto non viene mai rimosso, e

va bene così: a quel punto la partita è finita comunque, e chi ha chiamato il metodo lo capisce dal valore di ritorno.

9. Collisioni

La gestione delle collisioni viene effettuata nel metodo `handle_collisions()` della classe `Game`, chiamata a ogni frame dopo l'aggiornamento di bombe e nemici.

Vengono gestiti diversi tipi di collisione:

- Giocatore e porte: il contatto con una porta permette di accedere al livello precedente o successivo
- Giocatore e item: se il giocatore si trova sulla stessa cella di un item attivo, questo viene raccolto e viene applicato il relativo effetto
- Esplosioni e muri: un'esplosione distrugge i muri distruttibili e il giocatore riceve il relativo punteggio (1 per muro); inoltre può essere generato un item nella cella interessata
- Esplosioni e nemici: un nemico raggiunto da un'esplosione viene ucciso e il giocatore riceve il relativo punteggio (5 per Walker, 10 per Roamer e Chaser); inoltre può essere generato un item nella cella interessata
- Esplosioni e bombe: se un'esplosione raggiunge una bomba che non è ancora esplosa, viene innescata una reazione a catena
- Giocatore e nemici/esplosioni: se il giocatore entra in contatto con un nemico o con un'esplosione, perde una vita, viene riportato alla sua posizione iniziale e il livello corrente viene resettato (a meno che non abbia terminato le vite: in quel caso la partita termina)

La gestione è suddivisa tra le classi `Game` e `Level`: `Game` interroga `Level` sullo stato corrente e agisce di conseguenza.

10. Punteggio

I punti si prendono così:

- 1 per ogni muro distruttibile abbattuto
- 5 per un Walker
- 10 per un Roamer o un Chaser
- 5 dagli item punteggio
- a fine partita, se hai vinto, i secondi che ti restano

Il Chaser vale il doppio del Walker perché ti insegue e ucciderlo è molto più rischioso, mentre il Walker va dritto finché non sbatte e in pratica lo prendi quando vuoi. Il bonus tempo finale è richiesto dalla traccia.

Cosa succede quando muori

Qui c'era un buco che ho scoperto giocando. Quando perdi una vita il livello si resetta: i muri tornano su, i nemici pure. Ma il punteggio no.

Quindi potevi rompere trenta muri, farti ammazzare apposta, ritrovarti i muri di nuovo lì e rifarlo all'infinito. Ho provato e in un paio di minuti ero a qualche centinaio di punti sul primo livello senza aver mai finito niente.

La soluzione è un secondo campo, `level_score`, che salva il punteggio nel momento in cui entri in un livello. Quando muori, `score` torna a quel valore. Il punteggio si comporta come il livello: tutti e due tornano a com'erano quando sei entrato. Detta così sembra ovvia, ma non ci avevo pensato finché non ho visto il numero salire mentre morivo.

11. Classifica

Tre metodi dentro la classe `Leaderboard`: salvare, caricare, liberare. All'inizio li avevo fatti statici, perché la classe non ha nessuno stato da tenere: è solo un raggruppamento di funzioni che lavorano sullo stesso file. Poi li ho tolti perché il progetto chiede di usare le classi come classi, e una classe fatta solo di metodi statici in pratica è un namespace travestito. Adesso chi la usa si crea un'istanza e chiama i metodi su quella.

Il file

Una riga per partita, formato `<nome>;<punteggio>`. Il separatore è il punto e virgola e non lo spazio, così uno può scrivere "Mario Rossi" senza rompere tutto. Il file si chiama `leaderboard.txt`, percorso relativo e senza barre: così funziona uguale su Windows e su Linux, che era una delle cose su cui i docenti hanno insistito.

Salvare

Il metodo `save()` apre in `std::ios::app` e attacca una riga in fondo. Il file resta disordinato: l'ordinamento lo faccio in lettura.

Avevo pensato di tenere il file già ordinato, ma vuol dire rileggerlo tutto, inserire nel punto giusto e riscriverlo daccapo a ogni partita. Ordinare in lettura costa la stessa cosa e il salvataggio diventa istantaneo, quindi ho scelto quello.

Il nome lo scrivo carattere per carattere e sostituisco con un punto solo i caratteri che romperebbero il formato, cioè il punto e virgola e gli a capo. Gli spazi restano. Se il file non si apre il metodo esce e basta: il punteggio si perde ma il gioco non si pianta.

Caricare

Il metodo `load()` legge riga per riga e costruisce una lista concatenata già ordinata dal punteggio più alto al più basso. Non c'è nessuna funzione di ordinamento separata: ogni nodo lo infilo subito al posto giusto con `insert_sorted()`.

Quella funzione ha due casi. Se la lista è vuota, o se il punteggio nuovo batte quello in testa, il nodo diventa la nuova testa. Altrimenti scorro finché il prossimo ha ancora un punteggio maggiore o uguale, e mi fermo lì.

È dichiarata `protected`: la usa solo `load()` e non ha senso che qualcuno la chiami da fuori. Prima era una funzione statica a livello di file, fuori dalla classe; l'ho spostata dentro quando ho tolto gli static, così sta insieme al resto e la visibilità la gestisce la classe invece del file.

Leggere la riga senza string

Non possiamo usare string, quindi lo split me lo sono fatto a mano. Un ciclo trova la posizione del punto e virgola e la lunghezza della riga; se il separatore non c'è, o sta in prima posizione (nome vuoto), salto la riga e passo avanti.

Il nome lo copio carattere per carattere e lo tronco se è troppo lungo, mettendo il terminatore a mano. Il numero lo converto così:

```
int score = 0;
for (int i = sep_pos + 1; line[i] != '\0'; i++) {
    if (line[i] >= '0' && line[i] <= '9') {
        score = score * 10 + (line[i] - '0');
    }
    else {
        break;
    }
}
```

Il trucco è `line[i] - '0'`: in ASCII le cifre stanno una dopo l'altra, quindi la differenza tra il codice di un carattere numerico e quello dello zero è proprio la cifra. Il resto è la conversione posizionale normale, moltiplico per dieci a ogni passo.

La memoria

I nodi li alloco con `new`, quindi chi chiama `load()` deve poi chiamare `free_list()`. L'ho scritto anche nel commento sopra la dichiarazione, perché è il tipo di cosa che si dimentica. La schermata della classifica lo fa subito dopo aver stampato.

12. Menu

Tre voci: nuova partita, classifica, esci. Sono un enum, così il menu restituisce direttamente la scelta e chi lo chiama ci fa uno switch.

Disegnarlo

Il metodo `draw(WINDOW* win)` pulisce la finestra, ci disegna la cornice con `box()`, il titolo, una riga di separazione e le tre voci. Quella selezionata la evidenzio con `A_REVERSE`, che scambia i colori di testo e sfondo.

Lo sfarfallio

La prima versione era diversa e non funzionava bene. `draw()` creava la finestra con `newwin()`, disegnava e poi la distruggeva con `delwin()`, tutto a ogni tasto premuto. Il codice era più corto, ma il menu sfarfallava a ogni movimento della selezione.

Il problema erano due cose insieme. Creare e distruggere la finestra a ogni frame vuol dire che per un istante quella zona di schermo non appartiene a nessuna finestra. E leggevo l'input con `getch()`, che è `wgetch(stdscr)`: se `stdscr` risulta "sporco" `ncurses` lo rinfresca da solo, e siccome lì sotto non c'è il menu ma lo sfondo vuoto, me lo ridisegnava sopra.

Adesso la finestra la crea `show()` una volta sola prima del ciclo e la riusa, `stdscr` lo pulisco una volta all'inizio e poi non lo tocco più, e l'input lo leggo con `wgetch(win)` direttamente sulla finestra del menu. In più c'è un flag `need_redraw` che fa ridisegnare solo quando qualcosa è davvero cambiato, invece che a ogni giro del ciclo.

Muoversi tra le voci

Lo spostamento è circolare, dall'ultima voce torni alla prima. Salendo la riga è questa:

```
selected = (selected - 1 + MENU_ITEM_COUNT) % MENU_ITEM_COUNT;
```

Il "+ MENU_ITEM_COUNT" serve perché in C++ il modulo di un numero negativo resta negativo. Ero convinto che $-1 \% 3$ facesse 2 e invece fa -1, che rappresenta un indice non valido dell'array. Tale aggiunta non cambia il risultato, ma tiene l'operando positivo.

Leggere il nome

Il metodo `read_string()` legge una stringa da tastiera senza usare le funzioni di input della libreria standard. Spengo l'eco, accendo il cursore, e gestisco i tasti uno per uno: invio conferma, backspace cancella, i caratteri stampabili si aggiungono. Ogni carattere lo disegno io a schermo, visto che l'eco automatico è spento.

Il ; lo rifiuto direttamente in ingresso, oltre a filtrarlo in scrittura. È ridondante ma il file di testo qualcuno potrebbe aprirlo e modificarlo a mano.

Classifica e fine partita

Il metodo `show_leaderboard()` chiede quanti dei migliori vuoi vedere, come dice la traccia, carica la lista e stampa i primi N. Se scrivi qualcosa che non è un numero mette 10. Se non c'è nessun punteggio salvato scrive che non ce n'è, invece di lasciare una pagina vuota.

Il metodo `prompt_save_score()` parte a fine partita, sia se vinci sia se perdi. Mostra il punteggio, chiede il nome, salva. Se premi invio senza scrivere niente mette "Anonimo".

Il problema di nodelay

Su questo ho perso più tempo di quanto vorrei ammettere. Le due schermate mettono `nodelay(stdscr, FALSE)` quando entrano e lo rimettono a `TRUE` quando escono.

Il motivo è che il ciclo di gioco gira con `nodelay` attivo, altrimenti si bloccherebbe ad aspettare un tasto invece di aggiornare. Ma quella modalità è globale su `stdscr`. Quindi la schermata della classifica appariva e spariva nello stesso istante: `getch()` tornava subito con `ERR` senza aspettare niente, e il codice proseguiva.

Ci ho messo un'ora a capirlo perché cercavo l'errore nel disegno, non nell'input. Adesso ogni schermata che aspetta un tasto gestisce la propria modalità in entrata e in uscita.

Centrare il testo senza strlen

Tutte le lunghezze che mi servono per centrare le stringhe le calcolo con un ciclo che avanza fino al terminatore. Per la riga del punteggio finale, dove la lunghezza dipende anche da quante cifre ha il numero, c'è un metodo apposta, `digit_count()`. Ne parlo nella sezione sugli LLM, perché è venuto fuori da lì.

13. Grafica

Per la grafica abbiamo usato la libreria `ncurses`, che permette di disegnare caratteri sullo schermo del terminale.

Abbiamo deciso di rappresentare gli oggetti di gioco nel modo seguente:

- muro solido: spazio bianco
- muro distruttibile: spazio grigio (giallo su Windows)
- porte: < (per il livello precedente) e > (per il livello successivo), verdi
- bombe: O rossa (bianca se sta lampeggiando)
- esplosioni: * gialle su sfondo rosso
- giocatore: @ ciano
- nemici: X (Walker), ? (Roamer), ! (Chaser)
- item: R (Range), rombo (Life), sterlina (Score), T (Time)

Il game loop si occupa di chiamare il metodo `Renderer::render()` a ogni frame per mostrare gli aggiornamenti.

Colori su Windows

Questo è stato uno dei problemi più fastidiosi di tutto il progetto, e non aveva niente a che fare con la logica del gioco. Su Linux, `ncurses` ha 256 colori e per i muri distruttibili usavo un tipo di grigio della tavolozza estesa. Su Windows, dove si compila con `pdcurses`, i colori sono otto. Però gli indici oltre l'ottavo non danno errore: vengono semplicemente ignorati. Quindi i muri distruttibili sparivano dalla mappa, senza nessun messaggio, per cui per un po' ho creduto che fosse un bug della generazione.

La soluzione è un controllo a runtime su `COLORS` che ripiega su un colore base quando non ce ne sono 256. Stessa storia con `use_default_colors()`, che su `pdcurses` fallisce e faceva saltare tutte le coppie di colore che usavano lo sfondo trasparente: adesso controllo il valore di ritorno invece di darlo per scontato.

È esattamente il caso che i docenti avevano in mente quando hanno detto che il progetto deve girare su tutti e due i sistemi senza toccare il codice. Il file sorgente è uno solo, ma si adatta a quello che il terminale sa fare.

14. Implementazione per Windows

I docenti hanno chiesto che il progetto compili e giri sia su Windows sia su Linux senza modificare il codice sorgente. Io sviluppo su Windows con CLion, i tutor correggono su Linux: il rischio di scrivere qualcosa che funziona solo da una parte era concreto e in effetti ci sono cascato un paio di volte.

La regola che ho seguito è che le differenze tra i due sistemi devono stare tutte nella configurazione della build o in controlli fatti a tempo di esecuzione, mai in blocchi condizionali dentro il codice del gioco. Non c'è nessun `#ifdef _WIN32` nel progetto. Un `#ifdef` serve quando devi chiamare funzioni diverse su due sistemi diversi. Se per mettere in pausa il programma usi `Sleep()` su Windows e `usleep()` su Linux, allora ti serve un ramo condizionale:

```
#ifdef _WIN32
    Sleep(50);
#else
    usleep(50000);
#endif
```

Ma scegliendo `napms(50)`, che esiste in `ncurses` e in `pdcurses` con la stessa firma, il problema sparisce a monte. Stessa cosa per `clear()` invece di `system("clear")` e `getch()` invece della funzione di `conio.h`.

La libreria curses

Su Linux si installa `libncurses-dev` e si linka con `-lncurses`. Su Windows ho usato MinGW, che nei suoi pacchetti include una versione di `curses` (`pdcurses`) con lo stesso header e le stesse firme di funzione.

Questo è il motivo per cui il codice può restare identico: scriviamo `#include <ncurses.h>` e chiamiamo le sue funzioni allo stesso modo sui due sistemi. Quello che cambia è la libreria da linkare, ma quello lo decide CMake.

Il CMakeLists

Su Linux `find_package(Curses)` trova tutto da solo. Su Windows no, perché MinGW l'ho installato a mano e CMake non sa dove guardare: la configurazione falliva con "Could NOT find Curses".

```
find_package(Curses)

if(NOT CURSES_FOUND AND WIN32)
    set(CURSES_INCLUDE_DIR "C:/MinGW/include/ncurses")
    set(CURSES_LIBRARIES "C:/MinGW/lib/libncurses.a")
endif()

include_directories(${CURSES_INCLUDE_DIR})
target_link_libraries(BomberMan ${CURSES_LIBRARIES})
```

Il fallback è dentro un `if` con due condizioni: scatta solo se `find_package` fallisce e siamo su Windows. Su Linux quel blocco non viene mai eseguito, quindi il percorso fisso non dà nessun fastidio ai tutor. È anche il motivo per cui il percorso lo abbiamo scritto con le barre normali e non con quelle rovesciate: CMake le accetta anche su Windows e così il file non ha caratteri che su un altro sistema verrebbero letti come sequenze di escape.

Se qualcuno ha MinGW in un'altra cartella deve cambiare quelle due righe, ma è l'unico punto del progetto in cui compare un percorso assoluto.

Due modi per compilare

Nel repository ci sono sia un `CMakeLists.txt` sia un `Makefile`. Non è una duplicazione inutile: servono a due cose diverse.

- `CMakeLists.txt`: per aprire il progetto in CLion, come è stato richiesto per facilitare la correzione.
- `Makefile`: per compilare da terminale su Linux con i comandi `make`, `make clean`, `./bomberman`.

I colori, che invece si adattano da soli

Il caso dei colori è diverso dagli altri, perché non si può risolvere nella configurazione della build: dipende da cosa supporta il terminale su cui il gioco sta girando. Ne ho parlato nella sezione sulla grafica: `init_colors()` controlla a tempo di esecuzione quanti colori sono disponibili e ripiega su quelli base quando la tavolozza estesa non c'è.

È lo stesso principio degli altri punti, applicato in un momento diverso: un solo sorgente che si accorge da solo di dove sta girando.

Due cose che mi hanno fatto perdere tempo

La prima: su Windows non si può ricompilare finché BomberMan.exe è ancora in esecuzione. Il rebuild si ferma con “remove(BomberMan.exe): Access is denied”. Succede ogni volta che si chiude la finestra del terminale con la X invece di premere Q: il processo resta appeso e blocca il file. Si risolve chiudendolo da Task Manager, ma la prima volta non è per niente ovvio da capire.

La seconda: il gioco non va lanciato dal pulsante Run di CLion. La console integrata non è un terminale vero, ncurses legge dimensioni sbagliate e l'input si comporta male. Va aperto un terminale normale e dimensionato prima di avviare il gioco. Se il terminale è troppo piccolo il gioco si chiude e viene stampato un messaggio a schermo per avvisare l'utente, invece di disegnare solo una parte della mappa senza spiegare perché.

15. Utilizzo degli LLM (Simone Coppola)

Durante lo sviluppo del progetto ho utilizzato ChatGPT come strumento di supporto, evitando di affidargli direttamente la scrittura di codice. Mi è stato utile per confrontare diverse soluzioni, ottimizzare e migliorare la leggibilità del codice. In questo modo ho potuto mantenere una buona comprensione del programma e svilupparlo in autonomia.

Qui di seguito descrivo tre esempi concreti di utilizzo.

Ottimizzazione della gestione degli spawn dei nemici

Un esempio di utilizzo dell'IA riguarda la gestione delle posizioni disponibili per lo spawn dei nemici. In particolare mi è stata utile per ottimizzare la procedura di estrazione di una posizione casuale tra quelle disponibili.

L'implementazione iniziale prevedeva la scelta casuale di un indice tramite rand() % spawn_count. Dopo aver selezionato la posizione, questa veniva rimossa dall'array e gli elementi successivi spostati di una cella verso sinistra per mantenerli contigui.

Codice prima:

```
Position Map::get_random_spawn() {
    if (spawn_count <= 0) {
        return POSITION_NONE;
    }
    int i = rand() % spawn_count;
    Position spawn = spawns[i];
    for (int j = i; j < spawn_count - 1; j++) {
        spawns[j] = spawns[j + 1];
    }
    spawn_count--;
    return spawn;
}
```

Questo approccio funzionava correttamente, ma ogni estrazione aveva costo lineare rispetto agli spawn disponibili.

Ho chiesto all'IA di confrontare questo approccio con possibili alternative per rendere più efficiente l'estrazione delle posizioni casuali. Mi è stato suggerito di eseguire lo shuffle dell'array utilizzando l'algoritmo di Fisher-Yates e successivamente estrarre gli elementi dal fondo, senza effettuare spostamenti nell'array.

Ho quindi modificato l'implementazione iniziale introducendo una funzione `shuffle_spawns()`, che viene chiamata una sola volta, dopo aver salvato le posizioni disponibili. Così facendo la funzione `get_random_spawn()` può limitarsi a restituire l'ultimo elemento dell'array `spawns` e a decrementare `spawn_count`.

Codice dopo:

```
void Map::shuffle_spawns() {
    for (int i = spawn_count - 1; i >= 0; i--) {
        int j = rand() % (i + 1);
        if (j != i) {
            Position tmp = spawns[i];
            spawns[i] = spawns[j];
            spawns[j] = tmp;
        }
    }
}

Position Map::get_random_spawn() {
    if (spawn_count <= 0) {
        return POSITION_NONE;
    }
    else {
        Position spawn = spawns[spawn_count - 1];
        spawn_count--;
        return spawn;
    }
}
```

In questo modo lo shuffle dell'array ha costo lineare e viene effettuato una sola volta, mentre ogni successiva estrazione ha costo costante.

Quindi in questo caso l'IA mi è stata utile per migliorare l'efficienza di una parte di codice. Ho scelto di adottare la soluzione consigliatami basata sull'algoritmo di Fisher-Yates e l'ho integrata autonomamente nel progetto.

Riorganizzazione della classe Enemy

Un altro esempio di utilizzo dell'IA riguarda il miglioramento della gestione dei tre tipi di nemici. L'implementazione iniziale prevedeva una sola classe `Enemy`, che utilizzava un enum che indicava il tipo di nemico per distinguere `Walker`, `Roamer` e `Chaser` durante l'aggiornamento.

Di conseguenza, all'interno della classe erano presenti funzioni specifiche per ciascun tipo, come `move_walker()`, `move_roamer()` e `move_chaser()`, oppure `distance()` e `sort_directions()`, utilizzate solo da `Chaser`.

Questo approccio funzionava e mi permetteva di usare un solo array `Enemy enemies[MAX_ENEMIES]` nella classe `Level`, ma rendeva la classe `Enemy` responsabile di comportamenti molto diversi tra loro.

Codice prima:

```
enum EnemyType {
    ENEMY_WALKER,
    ENEMY_ROAMER,
    ENEMY_CHASER
};

class Enemy {
protected:
    Direction directions[DIRECTIONS_COUNT];
    Position p;
    Position start_p;
    int speed;
    int start_speed;
    int move_timer;
    bool dead;
    EnemyType type;
    Direction direction; // per walker

    int distance(Position a, Position b);
    void shuffle_directions();
    void sort_directions(Position player_p);
    bool can_move_to(Map& map, Position next);
    void move_walker(Map& map);
    void move_roamer(Map& map);
    void move_chaser(Map& map, Position player_p);
    void move(Map& map, Position player_p);

public:
    Enemy(Position _p = POSITION_NONE, int _speed = 1, EnemyType _type =
    ENEMY_WALKER);
    Position get_position();
    void set_position(Position _p);
    int get_speed();
    void kill();
    bool is_dead();
    EnemyType get_type();
    void update(Map& map, Position player_p);
    void reset();
};
```

Quindi ho chiesto all'IA se fosse più opportuno sfruttare l'ereditarietà in questo caso. Mi ha consigliato di utilizzare l'ereditarietà, creando la classe base Enemy e le classi derivate Walker, Roamer e Chaser.

Inoltre ha suggerito l'uso dei costrutti virtual e override; in particolare, nella sua implementazione della classe Enemy vengono definite le funzioni:

- virtual void move(Map& map, Position player_p) = 0;
- void update(Map& map, Position player_p);

E ciascuna classe derivata definisce il proprio metodo move:

- void move(Map& map, Position player_p) override;

Ho deciso di sfruttare l'ereditarietà, come consigliato dall'IA ma di non utilizzare virtual e override, perché non sono stati trattati durante il corso. Però questa scelta ha avuto due conseguenze.

Innanzitutto, non utilizzando virtual e override, non ho potuto sfruttare il polimorfismo per definire un unico metodo update nella classe base che richiamasse automaticamente la versione appropriata di move. Ho quindi dovuto definire un metodo update in ogni classe derivata.

Inoltre, utilizzando il polimorfismo con virtual e override avrei potuto gestire i diversi tipi di nemico attraverso un unico array Enemy enemies[MAX_ENEMIES] (perché update è definita nella classe base). Non utilizzandoli, ho dovuto mantenere tre array separati (utilizzando per ciascuno il proprio update):

- Chaser chasers[MAX_CHASERS];
- Roamer roamers[MAX_ROAMERS];
- Walker walkers[MAX_WALKERS];

Codice dopo:

```
class Enemy {
protected:
    Direction directions[DIRECTIONS_COUNT];
    Position p;
    Position start_p;
    int speed;
    int move_timer;
    bool dead;
    void shuffle_directions();
    bool can_move_to(Map& map, Position next);

public:
    Enemy(Position _p = POSITION_NONE, int _speed = 1);
    Position get_position();
    void set_position(Position _p);
    int get_speed();
    void kill();
    bool is_dead();
    void reset();
};

class Walker : public Enemy {
protected:
    Direction direction;
    void move(Map& map);
public:
    Walker(Position _p = POSITION_NONE, int _speed = 1);
    void update(Map& map);
};
```



```

class Roamer : public Enemy {
protected:
    void move(Map& map);
public:
    Roamer(Position _p = POSITION_NONE, int _speed = 1);
    void update(Map& map);
};

class Chaser : public Enemy {
protected:
    int distance(Position a, Position b);
    void sort_directions(Position player_p);
    void move(Map& map, Position player_p);
public:
    Chaser(Position _p = POSITION_NONE, int _speed = 2);
    void update(Map& map, Position player_p);
};

```

Quindi in questo caso l'IA mi è stata utile per rendere il codice più modulare e farmi comprendere i vantaggi dell'ereditarietà, ma l'implementazione finale è stata adattata agli argomenti affrontati durante il corso.

Miglioramento del movimento del Chaser

L'ultimo esempio relativo all'utilizzo dell'IA che propongo riguarda il comportamento del Chaser. Come descritto in precedenza nel paragrafo "Movimento (Chaser)" della sezione "5. Nemici", il suo movimento è basato su un approccio greedy: a ogni passo viene scelta la direzione che permette di avvicinarsi maggiormente al giocatore. Questo comportamento funziona bene in presenza di pochi ostacoli, ma può portare il nemico a rimanere bloccato o a muoversi in loop avanti e indietro quando chiamato ad aggirare una quantità maggiore di muri.

Per cercare di migliorare questo comportamento ho descritto il problema all'IA, gli ho chiesto di analizzare l'implementazione esistente e se fosse possibile migliorarla, mantenendo comunque una soluzione relativamente semplice.

Tra le varie soluzioni vi era innanzitutto l'utilizzo di un algoritmo di ricerca come la BFS (Breadth-First Search), che avrebbe risolto il problema alla radice permettendo al Chaser di calcolare un percorso completo verso il giocatore.

Mi ha proposto anche soluzioni più semplici e coerenti con quanto avevo già fatto, basate sul mantenimento dell'approccio greedy con l'introduzione di alcune euristiche. Una di queste prevedeva di scegliere la direzione secondo il criterio greedy nella maggior parte dei casi e di introdurre occasionalmente una scelta casuale, per esempio con probabilità 80% e 20% rispettivamente. Un'altra consisteva nel memorizzare l'ultima posizione occupata dal Chaser, evitando di tornarci in presenza di altre mosse disponibili, così da cercare di impedire il movimento avanti e indietro.

Ho testato queste euristiche, ma alla fine nessuna delle due si è rivelata soddisfacente. Infatti, l'introduzione della componente casuale rendeva il Chaser meno pericoloso durante l'inseguimento e non eliminava realmente il problema dei loop, che continuavano a verificarsi, seppur in misura minore. Invece la seconda soluzione inizialmente sembrava funzionare, ma successivamente ho osservato che spesso non eliminava i loop, ma

semplicemente ne aumentava la dimensione, costringendo il Chaser a percorrere un numero maggiore di posizioni prima di tornare nella zona che aveva già visitato.

Questi test mi hanno fatto capire che il problema non poteva essere risolto in modo soddisfacente con semplici correzioni al comportamento greedy. Per ottenere un inseguimento realmente efficace sarebbe quindi necessario adottare un algoritmo di ricerca come la BFS, capace di considerare il percorso nel suo complesso invece di valutare esclusivamente la singola mossa successiva.

Tuttavia, ho scelto di non sostituire la mia implementazione greedy con una BFS, per mantenere un comportamento semplice e coerente con quanto affrontato durante il corso. Ho quindi utilizzato l'analisi dell'IA principalmente per comprendere il limite dell'approccio iniziale e le possibili alternative, senza utilizzare la sua soluzione.

16. Utilizzo degli LLM (Vladimir Covaci)

Ho usato sia Geminidi di Google, sia Claude di Anthropic, quasi sempre per farmi rileggere il codice piuttosto che per farmelo scrivere, o come un punto di partenza. Però spesso le volte in cui gli ho chiesto di produrre codice da zero il risultato era corretto, ma scritto in uno stile diverso dal nostro, e finivo per riscriverlo comunque. Rileggere invece funziona bene, perché trova cose che io avevo davanti agli occhi da settimane senza vederle. Oppure se non avevo proprio idea di come implementare una certa cosa, risultava come un ottimo assistente.

Qui sotto ho dei casi, con il codice prima, la domanda che ho fatto, cosa mi ha risposto e cosa ho tenuto.

1. Una libreria che non potevo usare

IL PUNTO DI PARTENZA

La schermata di fine partita centra la riga "Punteggio: N". Per sapere quanto è lunga usavo `snprintf`. Rileggendo l'elenco delle librerie ammesse mi sono accorto che `<stdio>` non c'era. Era lì da mesi e nessuno se n'era accorto.

PRIMA

```
char score_str[32];
int score_len = snprintf(score_str, sizeof(score_str), "Punteggio: %d", score);
mvprintw(LINES / 2 - 2, (COLS - score_len) / 2, "%s", score_str);
```

COSA HO CHIESTO

"Nel progetto non posso usare `<stdio>`. Qui uso `snprintf` solo per sapere quanto è lunga la stringa da centrare a schermo. Come lo sostituisco rispettando i vincoli? Posso usare solo array di caratteri, niente string."

COSA MI HA RISPOSTO

Mi ha fatto notare che il buffer non serviva proprio. `mvprintw` è di `ncurses`, quindi ammesso, e sa già stampare un intero con `%d`: stavo costruendo una stringa per poi ristamparla identica. L'unica cosa che mi mancava davvero era la larghezza, che dipende da quante cifre ha il numero e si calcola dividendo per dieci finché non finisci. Mi ha proposto una funzione e la riscrittura della chiamata.

DOPO

```
// Numero di cifre decimali di un intero (0 -> 1).
int Menu::digit_count(int n) {
    if (n < 0) {
        n = -n;
    }

    int digits = 1;

    while (n >= 10) {
        n /= 10;
        digits++;
    }

    return digits;
}

// ...

const char* score_label = "Punteggio: ";
int score_len = 0;
while (score_label[score_len] != '\0') score_len++;
score_len += digit_count(score);
if (score < 0) score_len++;

mvprintw(LINES / 2 - 2, (COLS - score_len) / 2, "%s%d", score_label, score);
```

COSA NE PENSO

L'ho preso come stava. Oltre a togliere la libreria vietata sparisce anche un buffer da 32 byte che in teoria poteva troncarsi, e il calcolo manuale della lunghezza è lo stesso stile che usavo già in altri punti del file. Piccola cosa, ma è stato utile perché io stavo cercando "con cosa sostituisco snprintf" mentre la domanda giusta era se il buffer servisse.

2. Le esplosioni che si spegnevano tra loro

IL PUNTO DI PARTENZA

Gli avevo chiesto una rilettura generale di Map, senza avere in mente niente di preciso. Tra le cose che ha tirato fuori, una riguardava il campo explosion, che allora era un array di bool.

PRIMA

```
bool explosion[MAP_HEIGHT][MAP_WIDTH];

bool Map::is_explosion(Position p) {
    return !out_of_bounds(p) && explosion[p.y][p.x];
}

void Map::set_explosion(Position p) {
    if (!out_of_bounds(p)) explosion[p.y][p.x] = true;
}

void Map::unset_explosion(Position p) {
    if (!out_of_bounds(p)) explosion[p.y][p.x] = false;
}
```

COSA HO CHIESTO

"Rivedi la classe Map e dimmi se ci sono problemi che non ho notato. Il giocatore può avere fino a tre bombe attive contemporaneamente."

COSA MI HA RISPOSTO

Che il booleano non basta quando due esplosioni si sovrappongono. Se due bombe coprono la stessa cella e la prima finisce prima della seconda, lo spegnimento della prima azzerava il flag e quella cella smetteva di essere pericolosa, anche se la seconda stava ancora bruciando. Il giocatore ci passava dentro senza morire.

La proposta era di contare quante esplosioni coprono ogni cella invece di dire solo sì o no.

DOPO

```
int explosion[MAP_HEIGHT][MAP_WIDTH];

bool Map::is_explosion(Position p) {
    return !out_of_bounds(p) && explosion[p.y][p.x] > 0;
}

void Map::set_explosion(Position p) {
    if (!out_of_bounds(p)) explosion[p.y][p.x]++;
}

void Map::unset_explosion(Position p) {
    if (!out_of_bounds(p) && explosion[p.y][p.x] > 0) explosion[p.y][p.x]--; }
}
```

COSA NE PENSO

Accettato subito. La cella resta pericolosa finché c'è almeno una bomba che la copre, e il problema sparisce. È un conteggio dei riferimenti: liberi la risorsa solo quando l'ultimo che la usava la molla.

Questo caso mi ha colpito più degli altri. Il bug non dava errori di compilazione, non faceva crashare niente, e giocando non l'avrei mai trovato: capita solo se piazzavi due bombe vicine con tempi leggermente diversi, e il sintomo è che ogni tanto non muori quando dovresti. Una cosa che nel dubbio avrei dato alla fortuna.

C'è stato anche un effetto a catena. Con il contatore ogni incremento deve avere il suo decremento, se no il numero non torna. Questo ci ha costretto, nella classe Bomb, a salvare il raggio davvero raggiunto in ogni direzione invece di ricalcolarlo quando l'esplosione finisce. Nel frattempo i muri colpiti sono stati distrutti, quindi rifacendo il conto verrebbe un risultato diverso da quello dell'accensione e resterebbero celle accese per sempre. Invisibili, ma che ammazzano.

3. Metà mappa senza nemici

IL PUNTO DI PARTENZA

Stavo lavorando sulle zone sicure: volevo capire se aveva senso estenderle anche all'angolo destro, visto che da poco si rinasceva alla porta di ingresso. Gli ho chiesto di rivedere la generazione della mappa in generale, senza cercare niente di preciso. Il problema del bilanciamento è saltato fuori da lì e non me lo aspettavo: la funzione era una di quelle che avevo scritto mesi prima e non guardavo da allora.

PRIMA

```
bool Map::safe_zone(Position p) {  
    return p.y >= 0 && p.y < MAP_HEIGHT / 2 &&  
        p.x >= 0 && p.x < MAP_WIDTH / 2;}  
}
```

COSA HO CHIESTO

"Sto sistemando le safe zone della mappa. Rivedi come vengono generate le posizioni dei nemici e dimmi se vedi problemi."

COSA MI HA RISPOSTO

La prima cosa che ha fatto è stata contare le celle. La condizione ne escludeva 200 su circa 800, cioè un quarto della mappa: tutti i nemici finivano ammassati nelle altre porzioni e io potevo starmene nell'angolo in pace. Detto con un numero davanti è diventato ovvio, mentre a leggere il codice sembrava una condizione qualsiasi.

Poi ha aggiunto la cosa che stavo cercando io, ma da un'angolazione diversa: la zona protetta copriva una porta sola, mentre nel gioco si entra in un livello anche da destra. Ero partito per estendere le zone sicure e mi sono ritrovato a doverle anche rimpicciolire.

DOPO

```
const int SAFE_ZONE_SIZE = 5;  
  
bool Map::safe_zone(Position p) {  
    if (p.y < 0 || p.y >= SAFE_ZONE_SIZE) {  
        return false;  
    }  
  
    // Angolo sinistro  
    if (p.x >= 0 && p.x < SAFE_ZONE_SIZE) {  
        return true;  
    }  
  
    // Angolo destro  
    if (p.x >= MAP_WIDTH - SAFE_ZONE_SIZE && p.x < MAP_WIDTH) {  
        return true;  
    }  
  
    return false;  
}
```

COSA NE PENSO

Ho preso l'idea ma non la forma. Lui aveva scritto tutto in una condizione sola; io ho preferito spezzarla con le uscite anticipate, perché il controllo sulla riga vale per tutti e due i quadrati e così si fa una volta sola, e perché posso metterci sopra due commenti separati che dicono a cosa serve ognuno.

Le celle escluse sono passate da 200 a 50, e la zona adesso copre tutte e due le porte come volevo. Quello che mi resta di questo caso è che ero partito per aggiungere una cosa e sono finito a correggerne un'altra che non sapevo di avere: se non avessi chiesto una revisione generale, avrei esteso le zone sicure lasciando il quadrante da 200 celle dov'era, e il gioco sarebbe rimasto sbilanciato senza che me ne accorgessi.

In generale

La cosa che mi ha sorpreso di più è che i bug trovati nei casi 2 e 3 erano lì da settimane e non avevano mai fatto niente di visibile. Nessun errore di compilazione, nessun crash, solo comportamenti sbagliati ogni tanto che giocando non avevo notato. Se non li avessi cercati apposta sarebbero arrivati alla consegna.

Ho anche notato che la risposta cambia parecchio a seconda di quanto contesto gli dai. "Rivedi questo codice" produce osservazioni generiche che valgono per qualsiasi programma. Se invece scrivo che non posso usare certe librerie, o che le bombe attive possono essere tre, le osservazioni diventano specifiche e verificabili. Le due volte in cui ho ricevuto la risposta più utile è perché avevo messo nella domanda un vincolo concreto.

Ogni proposta l'ho comunque controllata prima di tenerla. Una l'ho modificata, una l'ho scartata dopo averla scritta tutta.